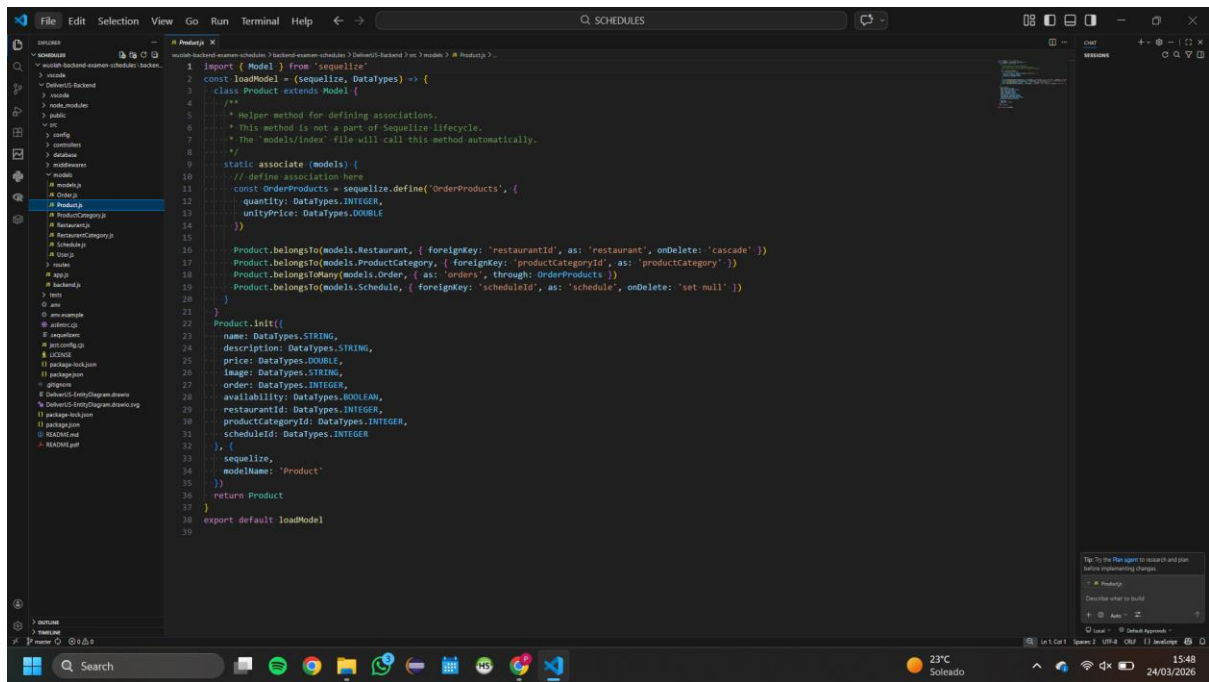


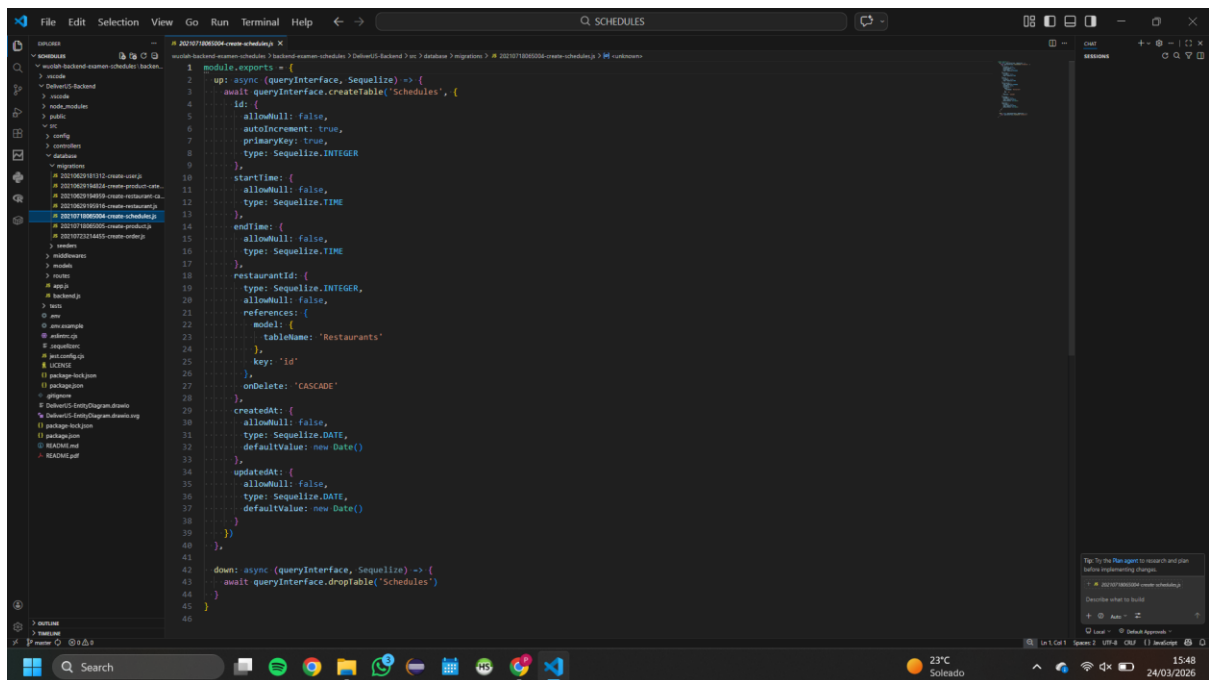
MODELS

```
File Edit Selection View Go Run Terminal Help Q SCHEDULES
model-backend-examen-schedule > backend-examen-schedule > Database-Backend > src > models > Schedule > .
1 import { Model } from 'sequelize'
2
3 const loadModel = (sequelize, DataTypes) => {
4   class Schedule extends Model {
5     /**
6      * Helper method for defining associations.
7      * This method is not a part of Sequelize lifecycle.
8      * The 'models/index' file will call this method automatically.
9      */
10    static associate(models) {
11      // Un horario pertenece a un restaurante
12      Schedule.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })
13
14      // Un horario puede tener varios productos asociados
15      Schedule.hasMany(models.Product, { foreignKey: 'scheduleId', as: 'products' })
16    }
17
18    Schedule.init({
19      startTime: {
20        allowNull: false,
21        type: DataTypes.TIME
22      },
23      endTime: {
24        allowNull: false,
25        type: DataTypes.TIME
26      },
27      restaurantId: {
28        allowNull: false,
29        type: DataTypes.INTEGER
30      },
31      createdAt: {
32        allowNull: false,
33        type: DataTypes.DATE,
34        defaultValue: new Date()
35      },
36      updatedAt: {
37        allowNull: false,
38        type: DataTypes.DATE,
39        defaultValue: new Date()
40      }
41    }, {
42      sequelize,
43      modelName: 'Schedule'
44    })
45  }
46  return Schedule
47}
```

```
File Edit Selection View Go Run Terminal Help Q SCHEDULES
model-backend-examen-schedule > backend-examen-schedule > Database-Backend > src > models > Restaurant > .
1 import { Model } from 'sequelize'
2 import moment from 'moment'
3
4 const loadModel = (sequelize, DataTypes) => {
5   class Restaurant extends Model {
6     /**
7      * Helper method for defining associations.
8      * This method is not a part of Sequelize lifecycle.
9      * The 'models/index' file will call this method automatically.
10     */
11     static associate(models) {
12       // define association here
13       Restaurant.belongsTo(models.RestaurantCategory, { foreignKey: 'restaurantCategoryId', as: 'restaurantCategory' })
14       Restaurant.belongsTo(models.User, { foreignKey: 'userId', as: 'user' })
15       Restaurant.hasMany(models.Product, { foreignKey: 'restaurantId', as: 'products' })
16       Restaurant.hasMany(models.Order, { foreignKey: 'restaurantId', as: 'orders' })
17       Restaurant.hasMany(models.Schedule, { foreignKey: 'restaurantId', as: 'schedules' })
18     }
19
20     async getAverageServiceTime () {
21       try {
22         const orders = await this.getOrders()
23         const serviceTimes = orders.filter(o => o.deliveredAt).map(o => moment(o.deliveredAt).diff(moment(o.createdAt), 'minutes'))
24         return serviceTimes.reduce((acc, serviceTime) => acc + serviceTime, 0) / serviceTimes.length
25       } catch (err) {
26         return err
27       }
28     }
29   }
30   Restaurant.init({
31     name: {
32       allowNull: false,
33       type: DataTypes.STRING
34     },
35     description: DataTypes.TEXT,
36     address: {
37       allowNull: false,
38       type: DataTypes.STRING
39     },
40     postalCode: {
41       allowNull: false,
42       type: DataTypes.STRING
43     },
44     url: DataTypes.STRING,
45     shippingCosts: {
46       allowNull: false,
47       type: DataTypes.DOUBLE
48     }
49   }, {
50     sequelize,
51     modelName: 'Restaurant'
52   })
53 }
```



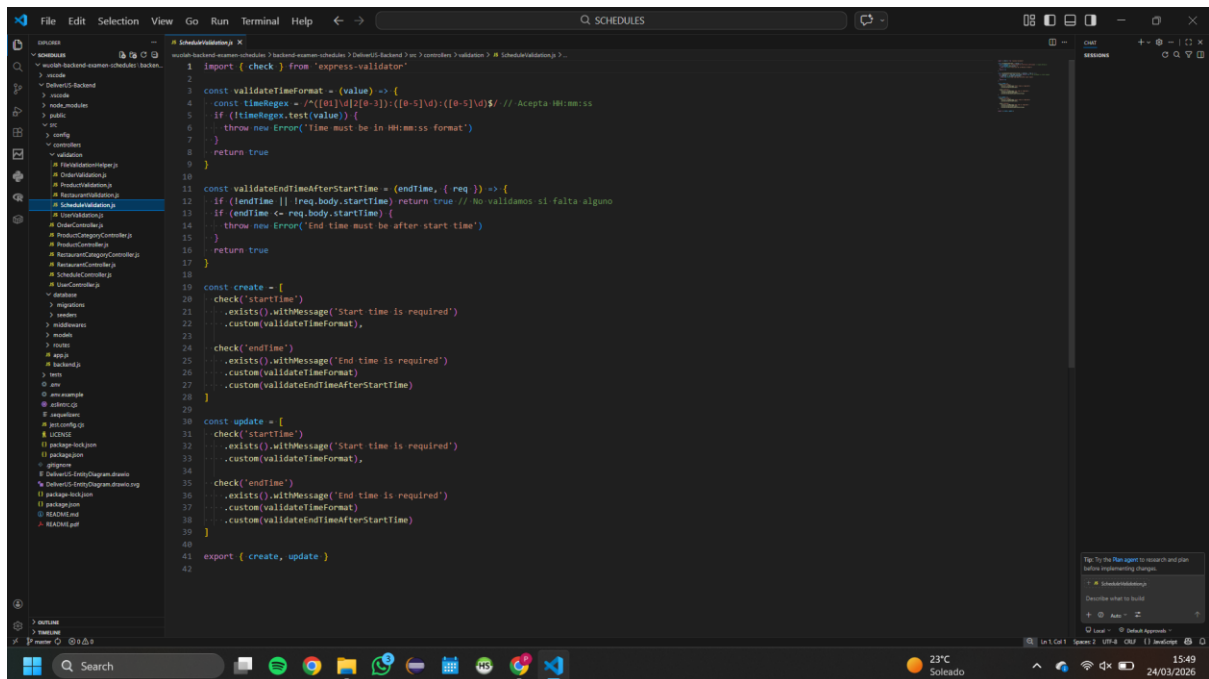
MIGRATIONS



CONTROLLERS

```
1 import { Schedule } from '../models/models.js'
2
3 const indexRestaurant = async function (req, res) {
4   try {
5     const schedules = await Schedule.findAll({
6       where: { restaurantId: req.params.restaurantId }
7     })
8     res.json(schedules)
9   } catch (err) {
10    res.status(500).send(err)
11  }
12 }
13
14 const create = async function (req, res) {
15   try {
16     const newSchedule = Schedule.build(req.body)
17     newSchedule.restaurantId = req.params.restaurantId // Asignar el restaurante correspondiente
18     const schedule = await newSchedule.save()
19     res.json(schedule)
20   } catch (err) {
21     res.status(500).send(err)
22   }
23 }
24
25 const update = async function (req, res) {
26   try {
27     await Schedule.update(req.body, { where: { id: req.params.scheduleId } })
28     const updatedSchedule = await Schedule.findById(req.params.scheduleId)
29     res.json(updatedSchedule)
30   } catch (err) {
31     res.status(500).send(err)
32   }
33 }
34
35 const destroy = async function (req, res) {
36   try {
37     const result = await Schedule.destroy({ where: { id: req.params.scheduleId } })
38     let message = ''
39     if (result === 1) {
40       message = 'Successfully deleted schedule id: ' + req.params.scheduleId
41     } else {
42       message = 'Could not delete schedule.'
43     }
44     res.json(message)
45   } catch (err) {
46     res.status(500).send(err)
47   }
48 }
49
50 const ScheduleController = {
51   indexRestaurant,
52   create,
53   update,
54   destroy
55 }
56
57 export default ScheduleController
```

VALIDATION



```
1 import { check } from 'express-validator'
2
3 const validateTimeFormat = (value) => {
4   const timeRegex = /^(01|02|03):([0-5]d):([0-5]d)$/ // Accepta HH:mm:ss
5   if (!timeRegex.test(value)) {
6     throw new Error('Time must be in HH:mm:ss format')
7   }
8   return true
9 }
10
11 const validateEndTimeAfterStartTime = (endTime, { req }) => {
12   if (!endTime || !req.body.startTime) return true // No validamos si falta alguno
13   if (endTime <= req.body.startTime) {
14     throw new Error('End time must be after start time')
15   }
16   return true
17 }
18
19 const create = {
20   check('startTime')
21     .exists().withMessage('Start time is required')
22     .custom(validateTimeFormat),
23   check('endTime')
24     .exists().withMessage('End time is required')
25     .custom(validateTimeFormat)
26     .custom(validateEndTimeAfterStartTime)
27 }
28
29 const update = {
30   check('startTime')
31     .exists().withMessage('Start time is required')
32     .custom(validateTimeFormat),
33   check('endTime')
34     .exists().withMessage('End time is required')
35     .custom(validateTimeFormat)
36     .custom(validateEndTimeAfterStartTime)
37 }
38
39 export { create, update }
```

ROUTES

